LABORATORY ACTIVITY III

AI-Assisted Frontend Framework Integration with an Existing REST API

Frontend Application Development for Information Technology

Continuation of Laboratory Activity II
Student Information Management System - Frontend Client for the Existing REST API

Laboratory Focus	Requirement
Development Mode	Frontend only - consume the REST API completed in Laboratory Activity I
Frontend Framework	Student choice using any appropriate client-side frontend framework/library
Backend	Use the student's existing framework-based REST API; no replacement backend
Primary Output	Working frontend application integrated with authenticated REST endpoints
Development Approach	AI-assisted development with human review, testing, and technical accountability

The frontend framework is a tool. Assessment focuses on client architecture, API integration, usability, state and error handling, security awareness, testing, and the ability to explain and modify the solution.

1. Laboratory Description

This laboratory activity is the direct continuation of Laboratory Activity II: AI-Assisted Framework-Based REST API Development. Students will retain the backend REST API they previously designed and implement a separate frontend application that consumes those existing API endpoints over HTTP.

The frontend must be developed using a client-side framework or library selected by the student. The completed application must authenticate against the existing backend, retrieve and manipulate real backend data, enforce role-aware navigation at the user-interface level, and provide professional data-management screens for the Student Information Management System.

This activity is intentionally frontend-focused. Students must not replace the REST API with local mock data, direct database access, server routes embedded in the frontend project, or a second backend. The backend created in Laboratory Activity I remains the authoritative source of data, validation, authentication, authorization, and business rules.

Core Continuation Rule

Laboratory Activity II must connect to and demonstrate the actual REST API developed in Laboratory Activity I. A visually complete application that operates only on hardcoded, static, or browser-only data does not satisfy this activity.

2. Learning Outcomes

Upon completion of the laboratory activity, students should be able to:

- Create a maintainable frontend project using a selected framework or library.
- Configure environment-based API endpoints and communicate with an external REST API.
- Implement authentication flows and protected application routes.
- Build reusable components, layouts, forms, tables, and navigation structures.
- Consume CRUD, search, filter, sorting, pagination, and domain-specific API endpoints.
- Manage loading, empty, success, validation, authorization, and network-error states.
- Implement role-aware user experiences without treating client-side checks as security boundaries.
- Apply responsive design, accessibility, usability, and consistent visual design principles.
- Test API integration and critical user flows.
- Use AI-assisted development tools productively while retaining ownership of architecture, code quality, and verification.

3. Relationship to Laboratory Activity I

Laboratory Activity I - Backend	Laboratory Activity II - Frontend
Exposes /api/v1 endpoints	Consumes /api/v1 endpoints
Authenticates users and issues session/token credentials	Provides login/logout UI and sends credentials appropriately
Validates requests and returns 4xx errors	Displays validation messages and preserves user input
Enforces roles and object-level authorization	Adapts visible navigation/actions to the authenticated role
Queries the relational database	Never connects directly to the database
Provides search/filter/sort/pagination metadata	Builds controls that send query parameters and render results
Returns structured JSON responses	Maps JSON responses to user-interface state
Produces authoritative academic data	Presents and edits that data through authorized workflows

4. Frontend Framework Freedom

Students may choose any appropriate frontend framework or client-side library. The selected framework will not determine the grade; implementation quality and understanding will.

Technology Family	Examples
JavaScript / TypeScript	React, Vue, Angular, Svelte, Solid, Preact
Meta-frameworks used as a client	Next.js, Nuxt, SvelteKit, or equivalent only when used as a frontend client and not as a replacement backend
Mobile-capable web UI libraries	Ionic or similar, when the output remains a client consuming the existing REST API
Other	Subject to instructor approval when it clearly functions as a frontend client

Pure Frontend Requirement

If a meta-framework supports server routes, server actions, ORM access, or database connections, those features must not be used to duplicate or bypass the existing REST API. All authoritative application data must come from Laboratory Activity I.

5. Recommended Frontend Architecture

```
Browser / Frontend Application
        |
        +-- Routes and Layouts
        +-- Pages / Views
        +-- Reusable Components
        +-- Forms and Validation
        +-- State / Query Management
        +-- API Client / Service Layer
                    |
                    v
          Existing REST API (Part I)
                    |
                    v
              Existing Database
```

6. AI-Assisted Development Policy

Artificial intelligence tools are encouraged throughout this activity as part of a modern software-development workflow. AI may accelerate implementation, but the student remains responsible for the correctness, security, maintainability, and explanation of all submitted work.

6.1 Appropriate Uses of AI

- Analyze the existing OpenAPI/Swagger specification or REST endpoint documentation.
- Plan frontend architecture, routing, components, state, and API-service boundaries.
- Generate framework scaffolding and repetitive UI structures.
- Create typed interfaces or data models from API responses.
- Draft forms, tables, reusable components, and client-side validation.
- Debug CORS, authentication, routing, state, and API integration issues.
- Generate unit, component, and end-to-end test cases.
- Review accessibility, usability, performance, and security concerns.
- Refactor code and improve naming, modularity, and consistency.
- Assist in README, setup, and technical documentation.

6.2 Required Human Verification

- Read and understand generated code before accepting it.
- Compare frontend assumptions with the actual backend API contract.
- Test successful and failed requests using real backend responses.
- Verify that role restrictions are still enforced by the backend.
- Remove unused, duplicated, insecure, or hallucinated code and dependencies.
- Confirm that the project builds and runs from documented setup instructions.
- Be prepared to explain, modify, or debug any important code during the demonstration.

Recommended AI Workflow

UNDERSTAND -> INSPECT API -> PLAN -> PROMPT -> GENERATE -> REVIEW -> INTEGRATE -> TEST -> DEBUG -> IMPROVE -> DOCUMENT

6.3 AI Development Log

Maintain a brief AI Development Log. The purpose is not to submit private reasoning; it is to document engineering use of AI.

Entry	Required Information
Task	What problem or feature was being addressed
AI Tool	Tool used, such as ChatGPT, Claude, Copilot, Cursor, Gemini, or another assistant
Prompt Summary	Short description of what was asked; full chain-of-thought is not required
Result	What code, design, test, or recommendation was produced
Human Review	What was accepted, changed, rejected, or verified
Evidence	Relevant commit, file, test, screenshot, or endpoint

7. Technical Requirements

Area	Minimum Requirement

Area	Minimum Requirement
Separate frontend project	Frontend source must be separate from the backend project, except when a monorepo clearly separates applications.
API configuration	API base URL must be configurable using environment variables or framework configuration.
Real API data	Core screens must load and modify data from the existing backend.
Routing	Use framework-appropriate routing for public, authenticated, and role-aware views.
Authentication	Login, logout, current-user state, and handling of unauthorized/expired sessions.
CRUD integration	Create, retrieve, update, and delete/deactivate records through the backend API.
Query features	Integrate search, filtering, sorting, and pagination where supported by the backend.
Forms	Use structured forms with clear validation feedback.
States	Provide loading, empty, success, error, forbidden, and not-found states.
Responsive UI	Application must remain usable on common desktop and mobile/tablet widths.
No direct DB access	The frontend must never connect to MySQL, PostgreSQL, SQL Server, SQLite, or another database directly.

8. Environment and API Configuration

The frontend must not hardcode production-specific URLs throughout the source code. Use framework-appropriate environment configuration.

```
# Example only - naming varies by framework
VITE_API_BASE_URL=http://localhost:8000/api/v1
# or
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000/api/v1
```

- Commit an .env.example or equivalent template, not private secrets.
- Do not place database passwords, JWT signing secrets, service-account secrets, or backend private keys in frontend code.
- If the backend is served from a different origin, configure CORS deliberately on the backend rather than disabling security globally.
- Document how to run the frontend and backend together during development.

9. Required Frontend Modules

The user interface must cover the major capabilities already provided by the backend REST API. Exact page names may vary by framework and design.

Module	Required Frontend Capabilities
Authentication	Login; logout; current-user identity; invalid-credential feedback; session/token expiry handling.
Dashboard	Summary cards or concise overview based on available backend data; quick navigation to major modules.
Students	List, search, filter, paginate, view details, create, edit, and delete/deactivate as permitted.
Programs	List and maintain academic programs based on authorized endpoints.
Courses	List and maintain courses; display code, title, units, status, and related details.
Academic Terms	List and manage academic year/semester records.

Module	Required Frontend Capabilities
Course Offerings	Create and manage course offerings with course, term, section, instructor, schedule, and capacity data.
Enrollments	Enroll students, view enrollment lists, update status, and prevent duplicate/error states through backend feedback.
Grades	Authorized grade entry/update and viewing of grade information.
Academic Record	Present the aggregated student academic-record endpoint in a readable format.
Profile / Account	Display authenticated user details and role; optional preference features are allowed.

10. Authentication and Protected Navigation

The frontend must integrate with the authentication mechanism implemented in Laboratory Activity I. The application should preserve authenticated state appropriately for the selected framework and provide a predictable response when authentication is missing, invalid, or expired.

- Public login page.
- Protected application shell/layout after successful authentication.
- Current-user request (for example, GET /api/v1/auth/me) when provided by the backend.
- Logout flow that clears client state and calls the backend logout/revocation endpoint when required.
- Automatic handling of 401 Unauthorized responses.
- Forbidden-page or equivalent handling for 403 responses.
- Route guards or protected-route mechanisms appropriate to the selected framework.

Security Boundary

Hiding a button or blocking a route in the frontend is not authorization. The backend from Laboratory Activity I must remain responsible for deciding whether an action is permitted.

11. Role-Aware User Experience

Role	Expected Frontend Behavior
Administrator	Navigation and controls for authorized system-management functions.
Registrar / Staff	Student, enrollment, academic-term, and other permitted management functions.
Instructor	Assigned offerings, enrolled students, and grade workflows that the backend permits.
Student	Own profile, own enrollments, own grades, and own academic record only.

The exact permission matrix should follow the backend implemented in Part I. Frontend behavior must not assume permissions that the backend does not grant.

12. API Client / Service Layer

Create a reusable mechanism for API requests. Depending on the framework, this may use fetch, Axios, Angular HttpClient, a query library, generated OpenAPI clients, or another appropriate approach.

- Central API base URL.
- Consistent authorization headers or credential settings.
- Common JSON parsing and error normalization.
- Handling for 401, 403, 404, 409, 422, and 500-class responses.
- Reusable functions/hooks/services for major resources.
- Cancellation or stale-request handling where useful.
- Avoid repeated raw endpoint strings and duplicated request boilerplate across components.

13. API Response and Error Handling

Backend Response	Frontend Expectation
200 / 201	Update UI state and show clear success feedback when appropriate.
204	Complete the action without expecting a JSON body.
400	Present a meaningful request error.

Backend Response	Frontend Expectation
401	Return the user to authentication or attempt the documented refresh flow.
403	Show an access-denied state; do not silently fail.
404	Show a not-found state or return to the resource list.
409	Explain the conflict, such as duplicate enrollment.
422	Map server-side field validation messages to the relevant form fields.
500+	Show a safe general error message and allow retry when appropriate.
Network failure	Show offline/unreachable-server feedback rather than an infinite loader.

14. User Interface and Experience Requirements

The application should have a professional, consistent, and readable interface. Students may use any design system or CSS approach, but the final application must demonstrate deliberate UI/UX decisions rather than default unstyled framework output.

- Consistent page layout, spacing, typography, colors, buttons, forms, dialogs, and tables.
- Clear navigation between major modules.
- Data tables with readable headers and predictable actions.
- Forms with labels, required-field indicators where useful, input help, and inline validation.
- Confirmation for destructive actions.
- Visible loading indicators or skeletons during API requests.
- Purposeful empty states when no records exist.
- Non-blocking success/error notifications where appropriate.
- Responsive behavior across common desktop, tablet, and mobile widths.
- Keyboard-accessible controls and semantic labels where practical.
- Avoid relying on color alone to communicate status.

15. Required Student List Integration

The Students screen is a required demonstration of collection-oriented API integration.

```
GET /api/v1/students?search=dela&program_id=1&year_level=3&page=2&per_page=20
```

- Search field linked to the backend search query.
- At least two backend-supported filters.
- Pagination controls driven by backend pagination metadata.
- Sorting when supported by the backend.
- Create-student form posting to the API.
- Edit-student form populated from backend data.
- Delete/deactivate action with confirmation.
- Validation messages from the API displayed to the user.

16. Forms and Validation

Client-side validation may improve usability, but it must not replace server-side validation. The frontend should interpret backend validation responses and associate them with the correct form fields.

Validation Layer	Purpose

Validation Layer	Purpose
Client-side	Immediate usability feedback, required fields, basic formats, preventing obviously incomplete submissions.
Backend	Authoritative validation, uniqueness, authorization, database relationships, and business rules.

17. State and Data Management

Students may choose the state-management strategy appropriate to the framework. A dedicated state library is not mandatory if built-in framework mechanisms are sufficient.

- Separate server/API state from temporary UI state where practical.
- Avoid unnecessary duplication of the same record in multiple disconnected stores.
- Refresh or invalidate stale lists after successful create/update/delete operations.
- Preserve user search/filter state when it improves usability.
- Do not treat browser storage as the authoritative database.

18. Suggested Frontend Project Structure

```
student-information-frontend/
|-- src/
|   |-- app/ or router/
|   |-- pages/ or views/
|   |-- components/
|   |-- features/
|   |-- services/ or api/
|   |-- hooks/ or composables/
|   |-- stores/ or state/
|   |-- types/ or models/
|   |-- utils/
|   `-- styles/
|-- public/
|-- tests/
|-- .env.example
|-- package.json (or framework equivalent)
`-- README.md
```

19. Laboratory Development Phases

Phase	Required Work
Phase 1 - Review Existing Backend	Run Laboratory Activity I, inspect Swagger/OpenAPI or Postman documentation, verify authentication, identify base URL and required endpoints.
Phase 2 - Frontend Setup	Select framework, initialize project, configure routing, environment variables, code quality tools, and base layout.
Phase 3 - API Client and Authentication	Create API service/client, login/logout, current-user state, protected routes, and 401/403 handling.
Phase 4 - Core Reference Data	Integrate Programs, Courses, and Academic Terms with list and authorized CRUD functions.
Phase 5 - Student Management	Integrate student list, details, search, filters, pagination, create, edit, and delete/deactivate.
Phase 6 - Academic Transactions	Integrate Course Offerings, Enrollments, and Grades.

Phase	Required Work
Phase 7 - Academic Record and Role Views	Render aggregated academic records and adapt navigation/actions for user roles.
Phase 8 - UX Hardening	Add loading, empty, validation, error, confirmation, responsive, and accessibility states.
Phase 9 - Testing	Perform component/integration/end-to-end tests and verify negative API scenarios.
Phase 10 - Documentation and Defense	Complete README, AI Development Log, evidence, build instructions, and prepare the live integrated demonstration.

20. Testing Requirements

Testing must include successful and unsuccessful interactions with the real backend. Students may use the testing tools preferred by their framework.

Test Area	Minimum Scenarios
Authentication	Successful login; invalid credentials; logout; protected route without authentication; expired/invalid session.
Students	Load list; search; filter; paginate; create; edit; duplicate/validation failure; delete/deactivate.
Authorization	Attempt an action hidden from or forbidden to a restricted role; confirm backend returns the expected denial.
Enrollments	Successful enrollment; duplicate/conflict; invalid reference; status change.
Grades	Authorized update; validation error; unauthorized update.
Networking	Backend unavailable; slow request/loading state; 500-class response.
Routing	Refresh on a nested route; not-found route; protected navigation.
Responsive UI	At least desktop and narrow/mobile viewport checks.

21. Automated Testing

Automated tests are strongly encouraged. Depending on the selected framework, students may use Vitest, Jest, Testing Library, Cypress, Playwright, Angular testing tools, or equivalent technologies.

- At least one authentication-flow test.
- At least one API-integrated list or component test.
- At least one form validation/error test.
- At least one protected-route or authorization-related test.
- At least one critical end-to-end flow when supported by the project setup.

22. Required Documentation

Document / Evidence	Minimum Content
README.md	Project purpose, framework, prerequisites, installation, environment configuration, run/build/test commands, API connection instructions.
API Integration Map	Frontend page/feature -> REST endpoint(s) -> HTTP method -> role/permission.
AI Development Log	Concise record of AI-assisted tasks and human verification.
Screenshots / Evidence	Major screens and important success/error states.

Document / Evidence	Minimum Content
Test Evidence	Automated test output and/or documented test cases.
Architecture Note	Short explanation of routing, API client, authentication state, components, and data/state strategy.

23. Required Final Outputs

1. Complete frontend source code.
2. Working connection to the existing Laboratory Activity I REST API.
3. .env.example or equivalent configuration template.
4. README.md with complete setup and integration instructions.
5. API Integration Map.
6. AI Development Log.
7. Automated tests and/or documented testing evidence.
8. Screenshots or equivalent evidence of major modules and error states.
9. Git repository with meaningful development history.
10. Working production build or framework-equivalent distributable output when applicable.

24. Mandatory Live Demonstration

The final demonstration must use the frontend and the actual backend together. The instructor may request additional modifications or troubleshooting.

#	Demonstration Task
1	Start or verify the existing REST API from Laboratory Activity I.
2	Start the frontend application and show its configured API base URL.
3	Attempt an invalid login and show error feedback.
4	Login successfully and show the authenticated user/role.
5	Navigate through a protected application route.
6	Load the Students list from the backend.
7	Search students using a backend query parameter.
8	Apply filters and pagination using backend-supported parameters.
9	Create a student and show the new backend record.
10	Trigger and display a backend validation error.
11	Edit an existing student.
12	Delete/deactivate a student with confirmation.
13	Load and manage Programs or Courses.
14	Load and manage an Academic Term and Course Offering.
15	Enroll a student through the API.
16	Encode/update a grade as an authorized user.
17	Display a student academic record from the aggregate endpoint.
18	Demonstrate a restricted role and a 403/authorization scenario.

#	Demonstration Task
19	Stop or disconnect the backend temporarily and show a controlled network-error state.
20	Show responsive behavior and explain the architecture, API service, authentication strategy, and AI-assisted workflow.

25. Minimum Acceptance Criteria

- Frontend starts without critical errors.
- Frontend and backend are separate logical applications.
- Actual Laboratory Activity I API is used for core data.
- Authentication and protected navigation work.
- Role-aware UI behavior is implemented.
- Student CRUD is integrated with real backend data.
- Programs, Courses, Academic Terms, Course Offerings, Enrollments, Grades, and Academic Record are represented in the frontend.
- Search, filtering, and pagination are integrated where supported by the backend.
- Backend validation messages can be displayed meaningfully.
- Loading, empty, network-error, unauthorized, forbidden, and not-found states are handled.
- Application is usable at desktop and narrow viewport sizes.
- No direct database connection exists in the frontend.
- No secrets or backend signing credentials are embedded in client code.
- Required documentation and test evidence are complete.

26. Grading Rubric - 100 Points

Criterion	Points	Full-Credit Indicators
REST API Integration	20	Real backend integration; centralized API handling; correct methods/parameters; reliable synchronization after mutations.
Authentication and Role-Aware Navigation	15	Login/logout, protected routes, session/token handling, 401/403 behavior, role-appropriate UI.
Core Functional Modules	15	Students, reference data, course offerings, enrollments, grades, and academic record function through the API.
Professional UI/UX	15	Consistent modern design, clear navigation, usable tables/forms, confirmations, feedback, responsive layout.
Forms, Validation and Error States	10	Client usability validation plus correct server validation display; loading/empty/network/error states.
Frontend Architecture and Code Quality	10	Reusable components/services, clear separation of concerns, framework conventions, readable maintainable code.
Testing and Reliability	5	Meaningful critical-flow tests and negative/error testing.
Documentation	5	Complete README, integration map, setup, architecture note, and evidence.
AI-Assisted Development and Technical Defense	5	Responsible AI use documented; student can explain, modify, and troubleshoot the implementation.
TOTAL	100	

27. Critical Deficiencies

Submissions with the following deficiencies will receive substantial deductions

No connection to the existing backend; hardcoded/static core data; direct database access from the frontend; frontend contains a replacement backend; authentication is simulated only in the UI; inability to demonstrate API requests; unhandled critical errors; secrets embedded in frontend code; copied or AI-generated implementation that cannot be explained or modified.

28. Bonus Enhancements

- Advanced accessibility improvements and automated accessibility tests.
- Optimistic updates with safe rollback.
- Advanced caching or query invalidation.
- Offline read-only support for previously fetched non-sensitive data.
- Charts or analytics based on backend data.
- Dark mode or theme system.
- Internationalization/localization.
- OpenAPI-generated typed API client.
- CI workflow for lint, test, and production build.
- Containerized frontend deployment or static hosting configuration.
- Performance profiling and bundle optimization.

Bonus features do not replace incomplete core requirements.

29. Technical Defense Questions

During the demonstration, the instructor may ask questions such as:

- How does the frontend know the backend base URL?
- Where is authentication state stored and why did you choose that approach?
- What happens when the backend returns 401, 403, 404, 409, or 422?
- Why is hiding a button not sufficient authorization?
- How does your student list implement backend pagination?
- How do you prevent duplicated API request logic across components?
- How do you refresh data after a successful create or update?
- Which validation belongs in the client and which belongs in the backend?
- What part of your project was AI-assisted, and how did you verify the output?
- If the backend changes an endpoint contract, where would you update the frontend?
- How would you diagnose a CORS error?
- How would you change one module if asked during the defense?

30. Final Integration Goal

Expected End State

Laboratory Activity I provides a secured, documented Student Information Management REST API. Laboratory Activity II adds an independently developed frontend framework application that authenticates against that API and presents the backend capabilities as a usable, professional information system. Together, the two activities demonstrate full client-server application development while preserving a clear separation between frontend and backend responsibilities.